

Going Live: What to Do to Run This App for Real

The app is **complete**: a cross-platform demand-to-outcome platform for creators — search-first archive, grounded cited answers, Demand Radar, the full agent layer (briefing, drafts, series, replies, loop-closing, sponsor reports, strategy advisor), connectors for Instagram / YouTube / TikTok plus newsletter / podcast / Discord demand sources, a creator Studio dashboard, billing with plans and quotas, and a hardened security/privacy posture. It's reviewed and tested (311 backend + 9 frontend tests).

What it is **not** yet is *live*: today it runs on a test machine with practice data and free **offline stand-ins** for the paid "AI brains," and platform accounts are connected by an operator rather than through a hosted sign-in screen. This document is the plain-language checklist for taking it from "runs locally" to "a real creator with real fans."

Steps are in rough order. Each is tagged: - **You** can do it yourself - **Developer** needed - **Legal/admin**

There's a one-paragraph **shortcut** at the bottom if you just want to see it working for real as fast as possible.

1. Get permission from the platforms (with some developer help)

The app only reads content and comments through each platform's official doors. It now speaks to more than Instagram, but Instagram is still the right place to start.

- [] Create a free **Meta developer account** at developers.facebook.com.
- [] Register the app and set up **Instagram Login** (no Facebook Page needed).
- [] Connect **one Instagram account you own or control** first. This is "Standard Access" — it works immediately, no review required, and is enough to prove the whole product end to end.
- [] Submit **App Review (Advanced Access)** to connect *other* creators later. Request only the permissions the demo visibly uses (extras are a common rejection reason). This can take days to weeks — **start it early**, in parallel.
- [] (*Optional, same model*) **YouTube** (Data API, owned/authorized channels) and **TikTok** (Display API — "archive" for content, "full-loop" where approved for comments/replies). The app already has working connectors for all three; each just needs its own API credentials.

Never scrape. Every platform detects and blocks it; Meta has taken legal action. Official APIs only — the app enforces this (it refuses to act on a capability an account hasn't been granted).

Permissions Instagram uses: `instagram_business_basic`, `instagram_manage_comments`, `instagram_manage_messages`, `_content_publish`.

The one remaining build gap. Today an operator pastes each account's access token in via an admin endpoint. A hosted **OAuth consent screen** ("Connect your Instagram" button) is the single piece not yet wired — fine for the first pilot (you connect the account yourself), needed before self-serve onboarding. Everything downstream of the token (sync, ingest, replies) is built.

2. Turn on the real "AI brains" (costs money)

Today every AI call is a free offline placeholder, so the app runs and tests with zero keys. For real quality, plug in paid providers — it's pasting keys into a settings file (see `.env.example`); each provider is swappable by config, never code:

- [] An **AI model** for answers, content ideas, drafts, and tagging (e.g. Anthropic or OpenAI). Set the latest capable model.
- [] An **embeddings** service that powers search.
- [] **Transcription** (spoken words) and **OCR** (on-screen text) — local for near-zero cost, or a paid service.
- [] (*Optional*) a **reranker** for sharper top results, and **Stripe** for real billing (both already behind interfaces with offline fakes).

Rough cost: tens of dollars to process one creator's entire back catalogue once, then cents per search after that. The app **meters AI spend per creator** and enforces a monthly cost cap per plan, plus a separate ceiling on the free public answer card so a fan page can't run up an unbounded bill — so costs stay predictable.

3. Put it on the internet

Move it off the test computer onto a real server. The app ships deploy manifests for **Kubernetes** and **Render** (`infra/deploy/`) plus `infra/docker-compose.yml`.

- [] Deploy the **API**, the **web app** (Qwik SSR), and the **workers** as standard containers.
- [] Stand up a real **PostgreSQL + pgvector** database, **Redis** (for the job queue and the scheduler), and **S3-compatible storage** (for downloaded media).
- [] **Run the database migrations** — `alembic upgrade head` (0001 → 0004). This builds the schema, including encrypted token columns and the connector tables.
- [] **Set the production secrets** — the app **refuses to boot** in production with insecure defaults, so this is not optional: `CCI_ADMIN_TOKEN` (operator auth), `CCI_ENCRYPTION_KEY` (encrypts stored OAuth tokens at rest), `CCI_IG_WEBHOOK_VERIFY_TOKEN` + `CCI_IG_APP_SECRET` (webhook signature — unsigned events are rejected in production), and `CCI_PSEUDONYM_SECRET` (keeps audience identities hashed, decoupled from the admin token). Set `CCI_DEBUG=false`.
- [] Buy a **domain** and serve over HTTPS. Each creator lives at `yourapp.com/<their-handle>`; the creator Studio dashboard sits behind operator auth.

- [] Point each platform's **webhook** at the live server so comment replies work.
- [] Make sure the **scheduler** is running — it drives hourly content sync, the DM dispatcher (which holds a per-creator lock so it can't double-send), the weekly Demand Radar build, the weekly digest, and daily token refresh.

Effort: a few days for someone who has done a containerized deployment before. The manifests, migrations, and health/metrics endpoints are already there.

4. Cover the legal basics (mostly done — review and publish)

You store what fans search and ask, so this is required (and Meta asks for it during App Review). Most of it is **already built** — your job is to review and publish.

- [] Publish the **privacy policy** and **terms of service** — drafts are written (docs/legal/PRIVACY.md , docs/legal/TERMS.md); have a lawyer review.
- [] The **deletion path** is built: GDPR export and erase (per creator, and erase-by-audience-pseudonym) are implemented endpoints.
- [] The app is privacy-safe by design: audience questions are stored **without names** (per-creator pseudonyms), PII is redacted from comment text before storage, and deleting a creator's post removes it from search and re-index.
- [] (Optional) turn on **row-level security** (shipped, opt-in) by running under a restricted DB role for defence-in-depth.

5. Decide what to charge (billing is built)

Plans, feature-gating, quotas, and cost caps already exist (free / creator / pro). To actually take money:

- [] Add **Stripe keys** (the billing provider is behind an interface with an offline fake; flip it to Stripe).
- [] Confirm the **plan boundaries** match the pricing you want — comment-to-DM automation, the draft generator, sponsor reports, and multi-platform ingestion are gated to the higher tiers; the public search + answer card is free. [↗](#)

This doesn't block a pilot — run the first creator on an internal/comp plan.

6. Run a real pilot

- [] Pick **one creator** who fits: 100+ evergreen posts, an audience that asks lots of repeat questions, and at least one thing to sell (course, product, affiliate).
- [] Connect their account(s), load their content (the sync runs automatically once connected — "Sync now" / "Sync all" in the Studio, or hourly on the scheduler), then have them spend **~2 hours labelling**

40–60 real questions (from their own comments) so the built-in **accuracy exam** can confirm it finds the right posts and cites them correctly.

- [] **Keep auto-DM in approve-first mode** — every reply gets a human OK — until the exam passes its accuracy and citation checks. (The exam now also fails closed if *nothing* is answerable, so a weak corpus can't sneak through.)
- [] Have the creator put the **search link in their bio**, add a permanent "Search" Highlight, and actually tell their audience to use it.
- [] Point the creator at the **Studio dashboard**: the opportunity queue (filterable by platform and by "gaps vs already-answered"), one-click draft / series / re-promote, the weekly briefing, and the made-vs-re-promoted loop count.

The two questions the pilot answers: 1. Can a real fan find the right old post **faster than through Instagram**? 2. Does the creator **act on Demand Radar** without being talked into it?

The shortcut (fastest path to "it works for real")

To see a real fan searching a real creator's archive and the creator acting on demand, you mainly need **steps 1, 2, and 3**: your own Instagram account connected (token pasted in), paid AI keys, and the app deployed with its secrets set and migrations run. Everything else — App Review for other creators, the hosted OAuth screen, Stripe, more platforms — is about *scaling* afterward, not proving it works.

What's already done (so you know what you're NOT paying to build)

The audience product - Hybrid search (full-text + vector + reranking) over captions, spoken words, and on-screen text, with grounded, **cited** answers and a "no strong answer" state. - The fan-facing search page (Qwik, light/dark, mobile-first, near-zero-JS). - Comment → drafted reply + DM, with human approval and the platforms' safety limits (one reply per comment, 7-day window, hourly cap), plus a per-creator dispatch lock.

Demand intelligence (the asset) - Demand Radar with **integrity** signals (unique askers, organic vs prompted, persistence, sentiment, manipulation-risk, exposure-normalized demand), an **explainable Opportunity Score**, and **reconciliation** (gap vs already-answered). - The north-star metric: closed loops per week, split **made vs re-promoted**. - The weekly **Briefing** digest content, content gap map, trend detection.

(The digest is fully rendered; its delivery transport is still a log stub — wiring it to email or DM is a small remaining job, below.)

The agent layer (what the creator pays for) - Content briefs, draft generator (script + hooks), creator recall search, multi-format repurposing + **Sift & Shift**, series builder. - Intent-labelled inbox, bounded clarifying questions, no-answer waitlist, saved playbooks, reply assistant, community delegation, "not now" deferral, **Loop-Closer**. - Affiliate injection + offer-aware CTAs + affiliate optimisation, sponsor reports, content strategy advisor, voice-consistency scoring; basic performance prediction, content calendar, thumbnails, pricing, and revenue forecasting.

Cross-platform - A connector framework with a capability registry; native connectors for **Instagram, YouTube, and TikTok** (TikTok archive vs full-loop per account); newsletter / podcast / Discord as demand sources; canonical-answer grouping across platforms; manual, all-at-once, and scheduled sync with per-connector "last synced" status.

Platform, security & ops - Swappable AI providers (offline fakes by default); creator-scoped data; OAuth tokens **encrypted at rest**; PII redaction + pseudonymisation; GDPR export/erase; webhook signature verification; **fail-closed boot** on insecure secrets; prompt-injection hardening on agent prompts; AI **cost metering + caps**; rate limiting; job retries. - Billing with plans, feature-gating, and quotas (Stripe behind an interface). - Alembic migrations (0001–0004), Kubernetes + Render deploy manifests, health/metrics endpoints, a built-in accuracy **exam** that gates auto-DM, and a full automated test suite (311 backend + 9 frontend), reviewed across five dimensions.

What's genuinely **left to build** is short: the hosted OAuth consent screen (one-click connect), the digest's **email/DM delivery transport** (the content is rendered today; only the sender is a stub), and the deliberately-deferred scale features (competitor radar, brand/marketplace portals, peer benchmarking, plagiarism monitoring, PM integration).

See `README.md` to run it locally, `docs/CURRENT_STATE.md` for the precise built-vs-not inventory, and `docs/MASTER_SPEC.md` for the full spec mapped to build status.